

COMP0264 Generative AI

Exam Revision Notes: Generative Adversarial Networks (GANs)

Auto-generated from Lecture Slides & Recordings

UCL 2025–26

Summary: This document covers all GAN-related content from COMP0264, including the basic GAN framework, the cross-entropy loss formulation, adversarial training, conditional GANs, training difficulties (mode collapse, vanishing gradients), and proposed solutions (Least-Squares GANs, instance noise, Wasserstein GANs). It also covers the PyTorch code tutorial for implementing a GAN on MNIST. Sources: Lectures 01-12, 01-13, 01-20; slides `GenAI2526_GANs.pdf` and `GenAI2526_GANCodeTutorialNotes.pdf`.

Contents

1 Overview	2
2 Generative Models: The General Framework	2
3 GAN Architecture	3
4 The GAN Loss Function	3
5 Adversarial Training	4
6 Using GANs After Training	5
7 Conditional GANs	5
8 Training Challenges	5
8.1 Mode Collapse	6
8.2 Vanishing Gradient Problem	6
9 Solutions to GAN Training Problems	7
9.1 Least-Squares GANs (LSGANs)	7
9.2 Instance Noise Technique	7
9.3 Wasserstein GANs (WGANs)	8
10 Summary of GAN Training Solutions	8
11 PyTorch Implementation: GAN on MNIST	8
12 Loss Landscape Visualisation	10
13 Key Connections and Broader Context	10
14 Potential Exam Questions	11

15 References**12**

Shuhao Wang

1 Overview

Generative Adversarial Networks (GANs) were the **first major topic** covered in COMP0264 (Lectures of 12 Jan, 13 Jan, 20 Jan 2026). GANs were introduced in a landmark paper at NeurIPS 2014 by Goodfellow et al. and were the dominant generative modelling paradigm from roughly 2014–2020.

LECTURE INSIGHT:

Why start with GANs? The lecturer chose GANs as the first topic because they provide a “nice example of generative systems” and serve as a conceptual warm-up. Although GANs are no longer the dominant paradigm (having been largely superseded by diffusion models and large language models), their principles remain foundational. The lecturer noted that there was a “cottage industry” of GAN variants proposed between 2014 and 2020, but a large repository collecting all these variants is now outdated (mentioned in Lecture 01-13).

2 Generative Models: The General Framework

Generative Models – Definition

Generative models use machine learning algorithms to **learn a distribution** of training data and then **generate new examples** from that distribution.

We model a generative model as a distribution $p(\mathbf{x} \mid \mathbf{w})$, where:

- $\mathbf{x}$ is a vector in the **data space** (e.g. pixel values of an image),
- $\mathbf{w}$ represents the **learnable parameters** of the model.

The problem is that in real-world applications, $p(\mathbf{x} \mid \mathbf{w})$ can be **extremely complex**, requiring a very large number of parameters. Deep learning provides powerful function approximators (neural networks) to model this distribution.

LECTURE INSIGHT:

Neural networks as function approximators The lecturer emphasised: “A neural network is a function. Sorry for destroying your dreams, but it is a function – and it is a very good function approximator.” Thanks to backpropagation and stochastic gradient descent (SGD), neural networks can optimise very complex functions with potentially trillions of parameters. The key conceptual point: a generative model is fundamentally a parametric probability distribution $p(\mathbf{x} \mid \mathbf{w})$ where $\mathbf{w}$ can be extremely high-dimensional.

LECTURE INSIGHT:

What does “generative” mean? Given a dataset $\mathcal{X}$, we assume observations were generated according to some unknown distribution p_{data} . The goal is to build p_{model} that can generate samples looking as if drawn from p_{data} . The model should **not simply memorise** the training data, but capture the underlying patterns – producing variations that are new yet plausible. “We are generating something new” (Lecture 01-12).

3 GAN Architecture

GAN Architecture

A GAN consists of two deep neural networks:

1. **Generator** $g(\mathbf{z}, \mathbf{w})$: Takes a latent vector $\mathbf{z}$ (sampled from a simple distribution, e.g. Gaussian) and maps it to the data space via a nonlinear transformation, producing synthetic data $\mathbf{x} = g(\mathbf{z}, \mathbf{w})$.
2. **Discriminator** $d(\mathbf{x}, \phi)$: A binary classifier that takes a data vector $\mathbf{x}$ and outputs the probability that $\mathbf{x}$ is real (as opposed to synthetic).

The discriminator's output uses a **logistic sigmoid** activation, so:

$$P(t = 1 | \mathbf{x}) = d(\mathbf{x}, \phi)$$

where $t = 1$ for real data and $t = 0$ for synthetic data.

LECTURE INSIGHT:

Adversarial nature The generator aims to **maximise** the discriminator's error (produce realistic images that fool the discriminator), while the discriminator tries to **minimise** the same error (become better at distinguishing real from fake). This creates a competition – “one network against the other”. The lecturer related this to **game theory**: it is a **zero-sum game** that, in the ideal case, reaches a **Nash equilibrium**. “If you move from that outcome, you will essentially not have a better result” (Lecture 01-12).

EXAM ALERT:

Job interview question: Why sigmoid in the discriminator? The lecturer explicitly flagged this as a typical **job interview question**: “Why does the discriminator use a sigmoid at the end?” Answer: The sigmoid maps the output to $[0, 1]$, which can be interpreted as the **probability** that the input is real. This is essential for the binary cross-entropy loss formulation. (Mentioned in Lecture 01-20. The lecturer also raised a subtle follow-up: “Is this a *real* probability?” – a question deferred to later discussion on calibration.)

4 The GAN Loss Function

Cross-Entropy Loss for GANs

The discriminator is trained using the standard **binary cross-entropy** loss:

$$E(\mathbf{w}, \phi) = -\frac{1}{N} \sum_{n=1}^N \{t_n \ln d_n + (1 - t_n) \ln(1 - d_n)\}$$

where N is the number of data points (e.g. in a mini-batch), $t_n \in \{0, 1\}$ is the target label, and $d_n = d(\mathbf{x}_n, \phi)$.

GAN-Specific Form of the Loss

For the specific case of a GAN, noting that for synthetic data $\mathbf{x}_n = g(\mathbf{z}_n, \mathbf{w})$, we can decompose the loss into sums over real and synthetic samples:

$$E(\mathbf{w}, \phi) = -\frac{1}{N} \sum_{n \in \text{real}} \ln d(\mathbf{x}_n, \phi) - \frac{1}{N} \sum_{n \in \text{synthetic}} \ln(1 - d(g(\mathbf{z}_n, \mathbf{w}), \phi))$$

LECTURE INSIGHT:

Derivation intuition When $t_n = 1$ (real), the second term $(1 - t_n) \ln(1 - d_n)$ vanishes, leaving only $\ln d_n$. When $t_n = 0$ (synthetic), the first term $t_n \ln d_n$ vanishes, leaving $\ln(1 - d_n)$. This is how the single cross-entropy splits into the two sums. “Think about it: if t_n is one, this goes to zero; if t_n is zero, this simplifies the other way” (Lecture 01-12).

5 Adversarial Training

Gradient Update Rules

The GAN can be trained **end-to-end** using stochastic gradient descent with backpropagation. The key point is the **opposite signs** in the update rules:

Discriminator (minimise error):

$$\Delta\phi = -\lambda \nabla_{\phi} E_n(\mathbf{w}, \phi)$$

Generator (maximise error):

$$\Delta\mathbf{w} = +\lambda \nabla_{\mathbf{w}} E_n(\mathbf{w}, \phi)$$

where $E_n(\mathbf{w}, \phi)$ denotes the error for data point $\mathbf{x}_n$ (or a mini-batch).

Note the different signs: the discriminator is trained to *decrease* the error; the generator is trained to *increase* the error.

LECTURE INSIGHT:

Training procedure in practice Training alternates between updating the generator and the discriminator on mini-batches. The lecturer described this as a **zero-sum game**: when one network improves, the other suffers. He emphasised the geometric intuition of gradient descent as navigating a loss landscape: “Think of a landscape with valleys – you want to reach the minimum. If λ is too big, you might jump over the valley; if too small, it takes forever” (Lecture 01-12).

STUDENT Q&A:

Can the learning rates differ for generator and discriminator? **Student:** Do the lambdas have to be the same for both?

Lecturer: “I don’t see any problem, because at the end you are essentially moving towards the good direction.” Different learning rates are acceptable – in practice this is common and can help balance training. (Lecture 01-12)

Ideal Outcome

If the generator finds a **perfect solution**, the discriminator will be unable to distinguish real from synthetic data, and its output will be $d(\mathbf{x}, \phi) = 0.5$ for all inputs. However, this 0.5 output can also arise pathologically (see mode collapse below).

6 Using GANs After Training

Inference with a Trained GAN

Once the GAN is trained:

1. The **discriminator is discarded**.
2. To generate new data: sample $\mathbf{z}$ from the latent space (e.g. $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$), then pass it through the trained generator: $\mathbf{x}_{\text{new}} = g(\mathbf{z}, \mathbf{w}^*)$.

LECTURE INSIGHT:

Pattern of discarding components The lecturer noted that this pattern – training with an auxiliary network and then discarding it at inference – appears in other generative models as well. “The discriminator was just used to train. We use only the generator network” (Lecture 01-12).

7 Conditional GANs

Conditional GANs (cGANs)

The GANs discussed so far are **unconditional**: they sample from $p(\mathbf{x})$.

Conditional GANs sample from a **conditional distribution** $p(\mathbf{x} | \mathbf{c})$, where $\mathbf{c}$ is a **conditioning vector** (e.g. a class label).

Implementation:

- Both the generator and discriminator take $\mathbf{c}$ as an **additional input**.
- After training, the desired class is selected by setting the corresponding value of $\mathbf{c}$.

Reference: Mirza & Osindero, arXiv:1411.1784 (2014).

LECTURE INSIGHT:

Why conditional GANs are efficient Instead of training 12 separate models for 12 cat breeds, a conditional GAN uses **the entire dataset** to learn shared internal representations (e.g. generic “cat” features), conditioned on the breed label. This is more efficient: “One model. Useful because we use the entire dataset” (Lecture 01-12).

The lecturer used the car analogy: instead of training separate models for Ferrari, Ford, etc., train one model that captures the general concept of “car” and conditions on make. “It’s more efficient because we have a bigger dataset” (Lecture 01-12).

8 Training Challenges

Overview of Training Difficulties

GANs are **not easy to train** due to their intrinsic adversarial nature:

- No standard error minimisation process – the objective can go up and down during training.
- Two competing objectives can lead to **oscillations** and **instability**.
- The major pathological case: **mode collapse**.

LECTURE INSIGHT:

Why training difficulty matters conceptually The lecturer stressed that understanding failure modes is critical for AI system design in general: “When you design an AI system, you have

many pathological situations, limit cases. Your system unfortunately ends up in these limit cases” (Lecture 01-13). He also noted: “We don’t really have a theory of deep neural networks yet – we know they work, but we don’t have closed-form results about why they work, why they generalise” (Lecture 01-13).

8.1 Mode Collapse

Mode Collapse – Definition

In **mode collapse**, the generator network’s weights adapt during training such that **all latent-variable samples are mapped to a small subset** of possible valid outputs. In extreme cases, the output can correspond to just one (or a small number of) output values $\mathbf{x}$.

Mechanism:

1. The generator produces only a small number of outputs that look somewhat realistic.
2. The discriminator cannot improve further and outputs 0.5 consistently (maximum uncertainty).
3. Since GAN training relies on the discriminator providing gradient feedback, the generator has **no gradient to improve further**, and training stalls.

STUDENT Q&A:

Mode collapse intuition **Student:** “Do we have to worry about the generator getting very good at generating a single image over and over again and not actually representing the distribution?”

Lecturer: “Exactly. That is the pathological case. We might become pathological. You get like one image or a subset of images, and then it becomes pathological.”

Another student phrased it well: “The goal is for [the generator] to represent the target distribution we care about. You might cheat by just getting really good at representing some subset of that.” – e.g. generating only the digit 3 when trained on MNIST. (Lecture 01-12)

EXAM ALERT:

Mode collapse – exam-critical concept Mode collapse is one of the most important GAN concepts. The lecturer discussed it across multiple lectures (01-12 and 01-13) and called it “the typical case” of GAN failure. Be prepared to:

1. Define mode collapse.
2. Explain *why* it happens (discriminator saturates at 0.5, no gradient signal).
3. Distinguish it from the vanishing gradient problem (a different failure mode).

8.2 Vanishing Gradient Problem

Vanishing Gradient in GANs

Consider a 1-dimensional example with a fixed data distribution $p_{\text{Data}}(x)$ and initial generator distribution $p_G(x)$ that are far apart.

The optimal discriminator $d(x)$ has **virtually zero gradient** in the vicinity of both $p_{\text{Data}}(x)$ and $p_G(x)$:

- Because $d(x) \approx 0$ across the region spanned by generated samples, small changes in the generator parameters $\mathbf{w}$ produce very little change in the discriminator output.
- Therefore the gradients are small $\Rightarrow$ **learning proceeds extremely slowly**.

Solution idea: Use a smoother version of the discriminator function $\tilde{d}(x)$ that provides a **stronger gradient signal**.

LECTURE INSIGHT:

Geometric intuition for vanishing gradients The lecturer used the “flat table” analogy: “If I am on this table and this table is part of my loss landscape, and the gradient is close to zero, I don’t know where to go.” When the two distributions are far apart, the discriminator’s decision boundary becomes a sharp step function with near-zero gradient everywhere except at the boundary – giving no useful signal to the generator.

He also showed the famous **loss landscape visualisations** (Li et al., NeurIPS 2018, “Visualizing the Loss Landscape of Neural Nets”) to illustrate how network architecture affects the loss surface. VGG-56 without skip connections has many flat areas and sharp local minima; with skip connections (ResNet), the landscape becomes much smoother.

Key takeaway: **Smoothness of the loss landscape** is crucial for trainability – not just in GANs but in deep learning generally. (Lectures 01-12, 01-13)

9 Solutions to GAN Training Problems

9.1 Least-Squares GANs (LSGANs)

Least-Squares GANs

Key idea: Modify the discriminator to produce **real-valued outputs** (not constrained to $(0, 1)$) and replace the cross-entropy loss with a **sum-of-squares error**.

This avoids the saturation of the sigmoid function and provides non-zero gradients even when the discriminator is confident.

Reference: Mao et al., ICCV 2017.

LECTURE INSIGHT:

Why sum-of-squares helps The standard sigmoid saturates at 0 and 1, producing near-zero gradients. A least-squares loss penalises deviations from the target value without saturation, providing a gradient signal everywhere. The lecturer described this as: “Instead of producing a probability in the range $(0, 1)$, we get a real number” (Lecture 01-13).

9.2 Instance Noise Technique

Instance Noise

Key idea: Add **Gaussian noise** to both the real data and synthetic samples during training. Adding noise smooths the data distributions, which in turn smooths the discriminator’s decision boundary. A smoother discriminator $\Rightarrow$ gradient is different from zero $\Rightarrow$ better training signal.

Reference: Sønderby et al., ICLR 2017. (The instance noise technique actually appears in the *appendix* of their paper on generative models with amortised variational inference.)

LECTURE INSIGHT:

Connection to PyTorch practice The lecturer noted that adding a Gaussian noise layer is standard practice in deep learning frameworks: “The Gaussian layer is available in PyTorch and TensorFlow. It just adds noise to the input.” The noise makes the sigmoid output imperfect (smoother), which ensures the gradient is non-zero. (Lecture 01-13)

Historical note: The paper by Sønderby et al. was actually about **super compression** at a startup (Magic Pony, acquired by Twitter), and the instance noise result appeared as an observation in the appendix. (Lecture 01-13)

9.3 Wasserstein GANs (WGANs)

Wasserstein GANs

Key idea: Instead of using cross-entropy, measure the **distance between the two distributions** $p_G(\mathbf{x})$ and $p_{\text{Data}}(\mathbf{x})$ directly.

The distance metric used is the **Wasserstein distance** (also known as the **Earth Mover’s Distance**):

- It measures how much “probability mass” needs to be moved to transform one distribution into another.
- It is differentiable and well-behaved, making it suitable for gradient-based optimisation.
- This is a fundamentally different approach from the standard adversarial loss.

Reference: Arjovsky et al., ICML 2017.

LECTURE INSIGHT:

Earth Mover’s Distance intuition The lecturer gave a vivid physical analogy: “You have two distributions. To make this one similar to that one, you need to physically *move* probability density from here to there – like moving earth. The Wasserstein distance measures how much you need to move.” (Lecture 01-13)

He also noted that many other distances could be used (e.g. KL divergence), but the Wasserstein distance has particularly nice properties for optimisation and computation. The WGAN approach is “completely different” from the standard adversarial loss: you are measuring distributional distance rather than playing a minimax game.

STUDENT Q&A:

Is the discriminator still used in WGANs? **Student:** “Is the discriminator used [in WGANs] like we set up just now?”

Lecturer: “This is a different approach. But actually, it is still [used] – this one is the loss for the quality of the generator.” In WGANs the discriminator (often called the “critic”) outputs an unbounded real value rather than a probability, and its architecture serves to approximate the Wasserstein distance. (Lecture 01-13)

10 Summary of GAN Training Solutions

Method	Core Change	Mechanism	Reference
Least-Squares GAN	Loss function	Replace cross-entropy with sum-of-squares; discriminator outputs real values, not probabilities	ICCV 2017
Instance Noise	Data preprocessing	Add Gaussian noise to real and synthetic samples, smoothing the distributions	ICLR 2017
Wasserstein GAN	Loss function	Use Earth Mover’s Distance between p_G and p_{Data} as the loss	ICML 2017

11 PyTorch Implementation: GAN on MNIST

This section summarises the code tutorial discussed in Lecture 01-20, where a GAN was implemented in PyTorch to generate MNIST digits.

PyTorch Training Loop – General Template

The standard training loop in PyTorch consists of five steps per batch:

1. **Forward pass:** `output = model(inputs)`
2. **Calculate loss:** `loss = criterion(outputs, labels)`
3. **Zero gradients:** `optimiser.zero_grad()` – clear gradients from the previous step
4. **Backward pass:** `loss.backward()` – compute gradients via backpropagation
5. **Optimiser step:** `optimiser.step()` – update weights

GAN-Specific Training in PyTorch

For a GAN, the training alternates between updating the **discriminator** and the **generator**:

Generator architecture: Sequential model with linear layers, batch normalisation, and LeakyReLU activations. The forward method takes noise **z** and outputs a generated image.

Discriminator architecture: Sequential model with linear layers, LeakyReLU, and a final **sigmoid** activation (to output a probability).

Loss: Binary Cross-Entropy (`nn.BCELoss`).

Training procedure:

1. Train discriminator: compute loss on real images (target = 1) and fake images (target = 0), then `backward()` and `step()`.
2. Train generator: generate fake images, compute discriminator loss on them with target = 1 (fool the discriminator), then `backward()` and `step()`.

Optimiser: Adam with appropriate betas (momentum parameters).

LECTURE INSIGHT:

Practical tips from the code tutorial

- **Batch normalisation** was used in the generator to prevent vanishing gradients.
- **LeakyReLU** was chosen over standard ReLU because it allows small gradients for negative values, preventing dead neurons.
- The code was run on Google Colab using CUDA GPUs. For Mac users, the MPS backend can be used instead.
- “After the first epoch, you start seeing some numbers appearing” – the GAN learns quickly on MNIST. (Lecture 01-20)

STUDENT Q&A:

Balancing generator and discriminator capacity **Student:** “If you make [the latent dimension] too big, hoping that your generator will be better, it turns out the discriminator gets too good too quickly, so you lose gradients. But if you make it too small, then your generator is bad. How do you find a middle ground?”

Lecturer: This is a fundamental challenge. In practice, you explore parameters through experimentation – there is no closed-form solution. Techniques include:

- **Grid search** over hyperparameters (learning rate, architecture size, activation functions).
- **Short training runs** to establish good hyperparameters before full training.
- Starting from values reported in the literature.
- Early stopping if training diverges.

“I know that is unsatisfactory, but in practice we don’t have any guarantee that certain values will have certain properties with the dataset that we have” (Lecture 01-20).

EXAM ALERT:

Hyperparameter selection as a “craft” The lecturer emphasised that selecting neural network hyperparameters is a “craft, not a science” – there is no theory to predict the best architecture or learning rate a priori for a given problem. For the exam, understand:

- Why grid search / random search is necessary.
- The distinction between a short “budget” training for hyperparameter selection vs. the full training.
- Why optimisation of neural networks is hard (composition of nonlinear functions $\Rightarrow$ non-convex landscape).

(Lecture 01-20)

12 Loss Landscape Visualisation

LECTURE INSIGHT:

Understanding loss landscapes The lecturer devoted significant time (Lectures 01-12, 01-13) to the concept of **loss landscape visualisation**, using work by Li et al. (NeurIPS 2018, “Visualizing the Loss Landscape of Neural Nets”).

Key observations from the visualisations:

- **VGG-56 without skip connections:** The landscape has sharp, narrow valleys and large flat plateaus. Getting stuck on a plateau means near-zero gradients and extremely slow learning.
- **VGG-110 without skip connections:** Even worse – larger flat areas, harder to optimise.
- **ResNet-56 with skip connections:** Dramatically smoother landscape – “smooth smoothing.” Skip connections provide additional gradient pathways, preventing the vanishing gradient problem.
- **DenseNet-121:** Also smooth, due to dense connectivity patterns.

Key takeaway: Skip connections and residual learning make the loss landscape smoother, which is why ResNets revolutionised deep learning. This applies to GAN training as well: smoother discriminator landscapes lead to better gradient signals.

Resources mentioned: <https://www.cs.umd.edu/~tomg/projects/landscapes/> and <https://losslandscape.com/explorer>

13 Key Connections and Broader Context

LECTURE INSIGHT:

Historical context The lecturer placed GANs in a broader historical context:

- The idea of machine-generated creative content goes back to the **Dartmouth Workshop proposal** (1955): “controlled randomness in otherwise orderly thinking” – essentially a prediction of probabilistic generative models.
- **Ada Lovelace** (1840s) wrote about machines that could “generate thinking, not only execute thinking” in her notes on Babbage’s Analytical Engine.
- **Herbert Simon** (1958): “There are now in the world machines that think, that learn, and that create.”
- The GAN paper (Goodfellow et al., 2014) was a major milestone, but the underlying ideas have deep roots.

LECTURE INSIGHT:

Discriminative vs. Generative models The lecturer contrasted the two paradigms:

- **Discriminative:** Given data and labels, learn a decision boundary (e.g. “is this a dog?” $\Rightarrow$ output 0 or 1).
- **Generative:** Learn the data distribution itself; then sample from it to create new data.

GANs cleverly use a discriminative component (the discriminator) to train a generative component (the generator).

14 Potential Exam Questions

EXAM ALERT:

Core concepts to know

1. **Define** a GAN and explain the roles of the generator and discriminator.
2. **Write out** the cross-entropy loss for a GAN and explain each term.
3. **Explain** how the gradient updates differ for the generator vs. discriminator (opposite signs).
4. **What happens** when a GAN reaches a perfect solution? (Discriminator outputs 0.5.)
5. **What is discarded** after training, and how do you generate new samples?

EXAM ALERT:

Mode collapse

1. Define mode collapse and explain the mechanism.
2. Why does the discriminator output 0.5 in mode collapse?
3. How does this differ from the “ideal” 0.5 output?
4. Give an example (e.g. generator produces only the digit 3 on MNIST).

EXAM ALERT:

Training difficulties and solutions

1. Explain the vanishing gradient problem in GANs using the 1D example.
2. Why does a smoother discriminator help? Relate to loss landscape visualisation.
3. Compare and contrast three solutions: LSGANs, instance noise, and WGANs.
4. What is the Earth Mover’s Distance and why is it suitable for GAN training?

EXAM ALERT:

Conditional GANs

1. How does a conditional GAN differ from an unconditional GAN?
2. Why is it more efficient to train one conditional GAN than multiple unconditional GANs?
3. How is the conditioning vector $\mathbf{c}$ incorporated into the architecture?

EXAM ALERT:

Job interview questions flagged by lecturer

1. Why does the GAN discriminator use a sigmoid activation function?
2. How do you select hyperparameters (learning rate, architecture) for training a neural network?
3. What is grid search and why is it necessary?
4. What role do skip connections play in making loss landscapes smoother?

(Explicitly flagged as interview-type questions in Lectures 01-13 and 01-20.)

15 References

Listed in Slides

- Chapter 19 of Bishop & Bishop, *Introduction to Deep Learning*, Springer 2024.
- Goodfellow et al., “Generative Adversarial Nets”, NeurIPS 2014.
- Mirza & Osindero, “Conditional Generative Adversarial Nets”, arXiv:1411.1784 (2014).
- Mao et al., “Least Squares Generative Adversarial Networks”, ICCV 2017.
- Sønderby et al., (Instance Noise), ICLR 2017.
- Arjovsky et al., “Wasserstein GAN”, ICML 2017.

Mentioned in Lectures (Additional)

- Li et al., “Visualizing the Loss Landscape of Neural Nets”, NeurIPS 2018.
Website: <https://www.cs.umd.edu/~tomg/projects/landscapes/>
- Loss Landscape Explorer: <https://losslandscape.com/explorer>
- Large collection of GAN variants (outdated but useful): mentioned as a GitHub repository of GAN implementations.
- McCarthy, Minsky, Rochester, Shannon, “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence”, 1955.
- Ada Lovelace, Notes on Babbage’s Analytical Engine (1843).
- Herbert Simon, “Heuristic Problem Solving” (1958).